



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВА РАБОТА

дисциплина: „Програмни среди”

**тема: „HCMSystemApp – уеб система за управление на човешки ресурси
с ASP.NET Core MVC, Entity Framework Core и ролеви достъп”**

Изготвили (III курс, специалност КСИ):

Петър Жан Иванов

Фак. № 121223054

42Б група

E-mail: petarzivanov@tu-sofia.bg

София, 2026

Съдържание

1. Въведение

2. Използвани технологии

2.1 ASP.NET Core MVC

2.2 Razor Views

2.3 Entity Framework Core

2.4 ASP.NET Core Identity

2.5 SQL Server / LocalDB

2.6 Repository Pattern

2.7 DataAnnotations

2.8 Dependency Injection

3. Какво беше необходимо, за да работи системата

3.1 Конфигурация на базата данни

3.2 Конфигурация на Identity

3.3 Seed данни

3.4 Одобрение на потребители

3.5 Ролева защита

4. Архитектура

4.1 Общ архитектурен модел

4.2 HCMSystemApp.Web

4.3 HCMSystemApp.Core

4.4 HCMSystemApp.Infrastructure

4.5 Поток на заявка в системата

5. Модел на базата данни

5.1 User

5.2 Role

5.3 Employee

5.4 Manager

5.5 Department

5.6 Salary

5.7 Payroll

5.8 Vacation

6. Основни функционалности

6.1 Регистрация и вход

6.2 Одобрение на нови потребители

6.3 Управление на служители

6.4 Управление на мениджъри

6.5 Управление на отдели

6.6 Payroll функционалност

6.7 Vacation функционалност

6.8 Ролева логика

7. Работни процеси

[7.1 Регистрация и вход](#)

[7.2 Одобрение на нов потребител](#)

[7.3 Управление на служители и отдели](#)

[7.4 Управление на мениджъри](#)

[7.5 Създаване и преглед на payroll](#)

[7.6 Заявяване и одобряване на отпуск](#)

[7.7 Обобщение на процесния модел](#)

8. Описание на реализацията

[8.1 Program.cs](#)

[8.2 DbContext](#)

[8.3 Repository](#)

[8.4 AccountService](#)

[8.5 PayrollService](#)

[8.6 VacationService](#)

[8.7 Controllers](#)

[8.8 ViewModels](#)

[8.9 Razor Views](#)

9. Примери за използване

[9.1 Нов потребител се регистрира](#)

[9.2 HR Admin одобрява потребител](#)

[9.3 Manager управлява служители от своя отдел](#)

[9.4 Manager създава payroll](#)

[9.5 Employee преглежда payroll](#)

[9.6 Employee заявява отпуск](#)

[9.7 Manager одобрява или отказва отпуск](#)

[9.8 Manager заявява отпуск и HR Admin го одобрява](#)

10. Екрани за демонстрация

11. Трудности и преодоленни проблеми

12. Заключение

13. Литература

1. Въведение

HCMSystemApp представлява уеб приложение за управление на човешкия капитал в организация. В текущата версия системата е реализирана като ASP.NET Core MVC приложение със слоеста архитектура Web/Core/Infrastructure. Това разделение позволява ясно разграничаване между потребителски интерфейс, бизнес логика и достъп до данни, което е особено важно при проект с роли, административни операции и различни потребителски сценарии.

Практическата идея е да се моделира реален HR процес. Нов потребител се регистрира, но не получава автоматично пълен достъп. Достъпът се активира след одобрение от HR администратор, който задава роля и при нужда организационни или финансови атрибути, например отдел и заплата. Така проектът следва логика, близка до реалните вътрешни системи, където контролът върху достъпа е задължителен.

Системата адресира няколко свързани проблема:

- централизирано управление на служители и мениджъри;
- структуриране на организацията чрез отделения;
- обработка на възнаграждения чрез Salary и Payroll;
- заявяване и одобряване на отпуски със статуси;
- ограничаване на видимостта и действията според ролята.

Поддържаните основни роли са Employee, Manager и HRAdmin. Служителят работи основно със собствените си данни, мениджърът управлява служители в рамките на отдел, а HR администраторът поддържа глобалните административни процеси. Тази роля-зависима структура прави проекта по-богат от обикновен CRUD пример.

Проектът е подходящ за курсова работа по „Програмни среди“, защото комбинира съвременни .NET технологии, релационна база данни, ASP.NET Core Identity, service/repository модел и реални бизнес сценарии. Освен работещ интерфейс, реализацията показва и архитектурно мислене при разделяне на отговорности.

2. Използвани технологии

2.1 ASP.NET Core MVC

ASP.NET Core MVC е използван като основна уеб рамка. Моделът Controller-View-Model улеснява разделянето между входящите HTTP заявки, визуализацията и данните, които се подават към потребителя. В HCMSystemApp контролерите приемат заявки, извикват service методи и връщат подходящи Razor изгледи или пренасочвания.

Този подход е удобен за HCM система, защото различните роли използват различни екрани и действия. MVC структурата позволява тези действия да се организират в отделни контролери и зони, без бизнес логиката да се натрупва във View слоя.

2.2 Razor Views

Потребителският интерфейс е реализиран с Razor Views. Изгледите работят с ViewModel-и и позволяват формите за вход, регистрация, payroll, отпуск и административни действия да бъдат ясно

свързани с controller actions. Чрез tag helpers като asp-for, asp-action и asp-controller се намалява количеството ръчно написан HTML код.

Сървърната валидация се визуализира чрез стандартните MVC validation механизми. Това е полезно при форми, в които има задължителни полета, дати, парични стойности и текстови ограничения.

2.3 Entity Framework Core

Entity Framework Core изпълнява ролята на ORM слой. Връзката с базата данни е реализирана чрез ApplicationDbContext, където се дефинират DbSet-и, relationship правила и конфигурации. LINQ заявките в service слоя позволяват филтриране, включване на навигационни свойства и проекция към ViewModel-и.

Проектът използва миграции, конфигурационни класове и seed данни. Това прави началното състояние на системата възпроизводимо и улеснява стартирането на проекта в локална среда.

2.4 ASP.NET Core Identity

ASP.NET Core Identity е използван за authentication и authorization. Проектът използва custom User и Role класове, което позволява стандартният identity модел да бъде разширен с домейн данни като име, възраст, статус на верификация и връзки към други бизнес записи.

Ролите HRAdmin, Manager и Employee са основа за достъпа до функционалности. Достъпът се контролира чрез Authorize атрибути и допълнително чрез service филтриране, например при показване на служители или pending отпуски.

2.5 SQL Server / LocalDB

Според конфигурацията в Program.cs е използван SQL Server provider чрез UseSqlServer. Конкретната база се определя от connection string DefaultConnection. В локална среда това може да бъде SQL Server Express или LocalDB, а при разгръщане може да се използва отделна SQL Server инстанция.

2.6 Repository Pattern

Достъпът до данни е абстрахиран чрез IRepository и Repository. Services използват общи операции като All, AllReadonly, AddAsync, GetByIdAsync, Delete и SaveChangesAsync. Това намалява директната зависимост от DbContext и държи бизнес логиката по-концентрирана.

2.7 DataAnnotations

DataAnnotations са използвани в entity и view model класовете за валидация и ограничения на данните. Атрибути като Required, MaxLength, ForeignKey и Column(TypeName = "decimal(18,2)") подпомагат както правилното създаване на schema, така и валидирането на входните данни.

2.8 Dependency Injection

Системата използва вградения dependency injection контейнер на ASP.NET Core. В Program.cs се регистрират DbContext, repository и основните services. Контролерите получават зависимостите чрез constructor injection, което прави кода по-четим и по-лесен за поддръжка.

3. Какво беше необходимо, за да работи системата

3.1 Конфигурация на базата данни

Приложението използва ApplicationDbContext и SQL Server provider. За създаване и актуализиране на schema се използват EF Core migrations. Конфигурациите на entity връзките и seed данните са организирани в отделни класове в Infrastructure слой, което прави DbContext по-чист и по-лесен за четене.

3.2 Конфигурация на Identity

Identity е конфигуриран с AddIdentity<User, Role>() и Entity Framework store към ApplicationDbContext. Вход, регистрация и изход се обработват през AccountController и свързаните view models. Ролевият достъп се прилага върху actions за служители, мениджъри и HR администратор.

3.3 Seed данни

В текущата версия на проекта са налични seed данни за начална демонстрация на системата:

- роли: HRAdmin, Manager, Employee;
- потребители: admin@example.com, manager@example.com, employee1@example.com и employee2@example.com;
- връзки между потребители и роли чрез UserRoleConfiguration;
- примерен manager, department и employee записи;
- salary, payroll и vacation записи за демонстрация.

Тези данни позволяват проектът да се стартира и демонстрира без ръчно въвеждане на цялата начална структура. Това е особено полезно при защита, защото могат да се покажат различните роли и потоци веднага след стартиране.

3.4 Одобрение на потребители

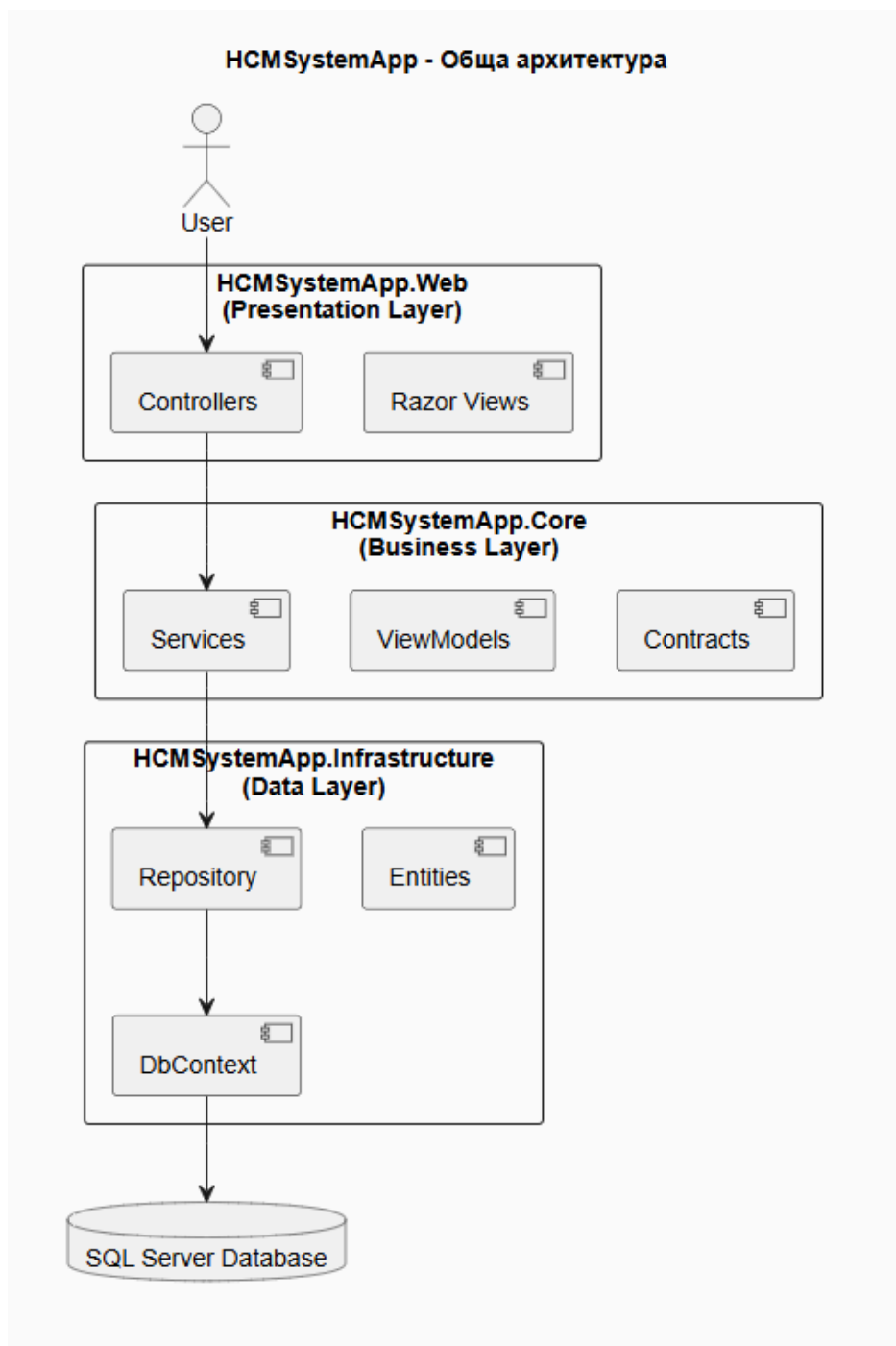
Нов потребител се регистрира, след което попада в административен процес по одобрение. HRAdmin преглежда чакащите потребители, задава роля и при нужда допълнителни данни като отдел и заплата. Така регистрацията не се третира като директен достъп до системата, а като начална стъпка от onboarding процес.

3.5 Ролева защита

Ролевата защита се реализира чрез комбинация от Authorize атрибути и проверки в service слой. Employee има достъп до лични данни, payroll и отпуски. Manager управлява служители в собствения си отдел и участва в одобрения. HRAdmin има достъп до административната зона, одобрение на потребители и управление на организационни структури.

4. Архитектура

Фигура 4.1: Обща архитектура на системата



4.1 Общ архитектурен модел

Проектът следва слоеста архитектура. Web слойът съдържа MVC контролери, Razor Views и UI логика. Core слойът съдържа service contracts, services, ViewModel-и и бизнес правила. Infrastructure слойът съдържа entity модели, ApplicationDbContext, repository и конфигурации на данните.

Това разделение намалява зависимостите и прави проекта по-лесен за разширяване. При добавяне на нова функционалност логиката може да бъде поставена в service слой, без да се смесва с HTML или директна работа с база данни.

4.2 HCMSystemApp.Web

HCMSystemApp.Web съдържа контролери като AccountController, PayrollController, VacationController, DepartmentController и HomeController. В проекта има и административна зона Areas/Admin, която отделя административните операции от основния потребителски поток.

UI е реализиран с Razor Views и Bootstrap/custom CSS. Този слой отговаря за форми, списъци, бутони, validation messages и пренасочване на потребителя към правилния екран.

4.3 HCMSystemApp.Core

В Core са дефинирани interfaces и service класове като IAccountService/AccountService, IPayrollService/PayrollService, IVacationService/VacationService и IDepartmentService/DepartmentService. Тук се намират и ViewModel-и като PayrollViewModel, VacationViewModel, VacationFormModel и DisplayedEmployeeModel.

Core не съдържа UI логика. Неговата задача е да приложи бизнес правила и да подготви данните във форма, удобна за визуализация.

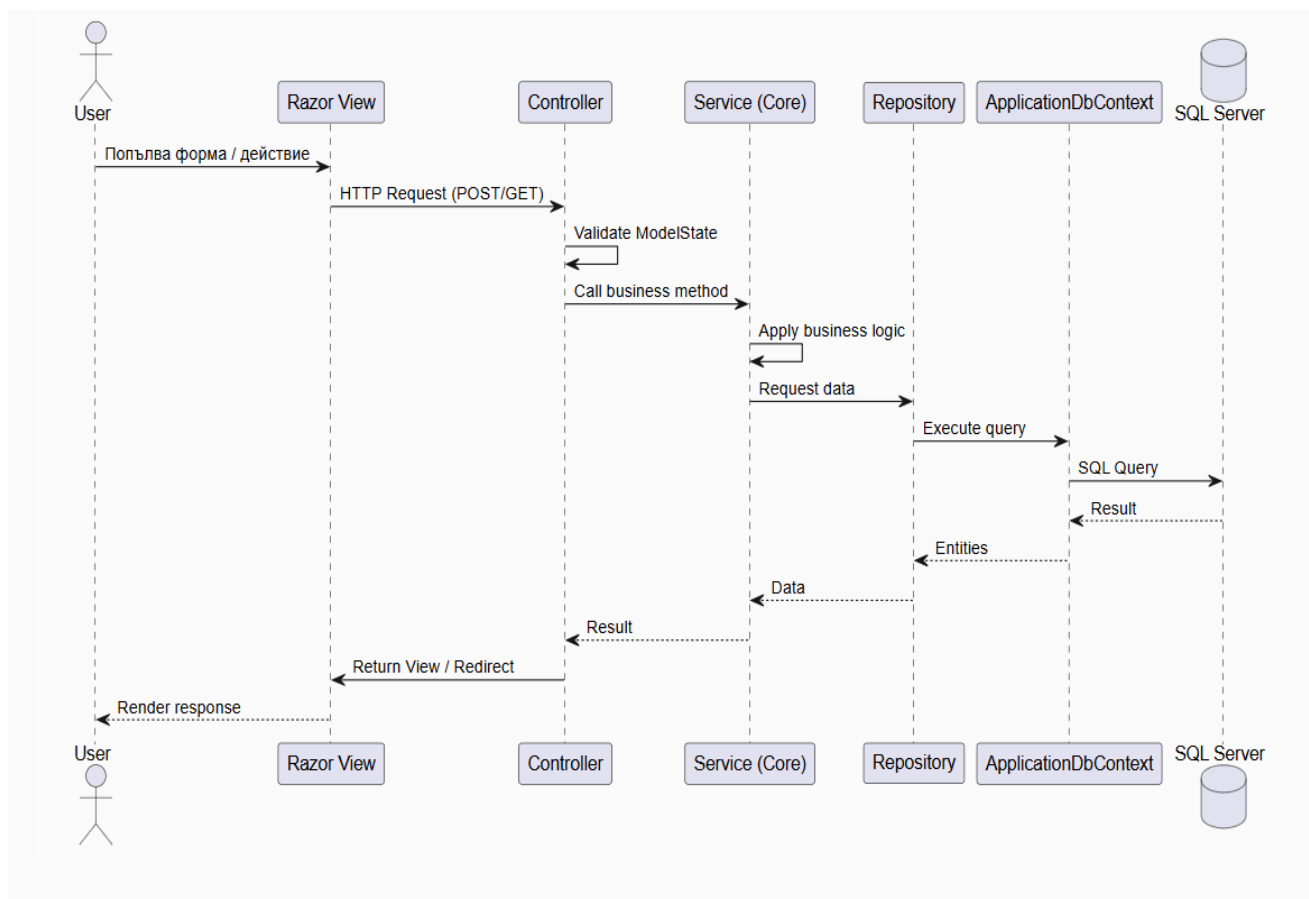
4.4 HCMSystemApp.Infrastructure

Infrastructure слойът съдържа entity класовете User, Role, Employee, Manager, Department, Salary, Payroll, Vacation и UserRole. Тук се намират ApplicationDbContext, Repository/IRepository, Fluent API конфигурациите, миграциите и seed данните.

4.5 Поток на заявка в системата

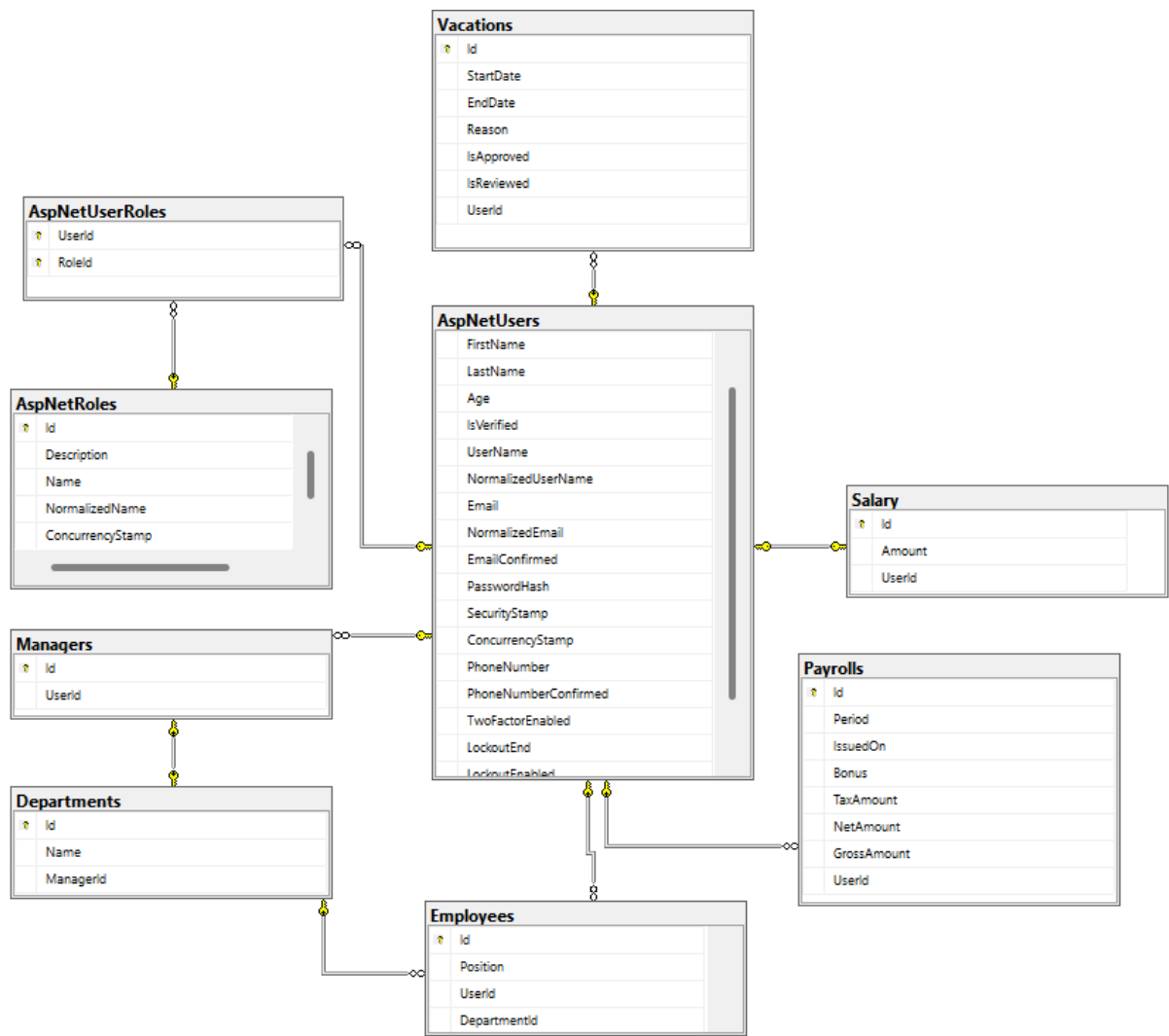
1. Потребителят изпраща форма или отваря страница от Razor View.
2. Controller action приема заявката и валидира входния модел.
3. Controller извиква service метод от Core слоя.
4. Service прилага бизнес правила и използва repository за данните.
5. Repository работи с ApplicationDbContext и SQL Server.
6. Резултатът се връща към controller, който визуализира View или прави redirect.

Фигура 4.2: Поток на заявка



5. Модел на базата данни

Фигура 5.1: ER диаграма на HCMSystemApp



Entity	Основна роля в системата
User / Role / UserRole	Identity и ролеви достъп
Employee / Manager	Организационни роли и връзки
Department	Структуриране на хората по отдели
Salary	Текуща база за възнаграждение
Payroll	Исторически платежни записи
Vacation	Заявки и решения за отпуск

5.1 User

User наследява Identity потребител и добавя бизнес полета като FirstName, LastName, Age, IsVerified и PhoneNumber. Чрез UserRole е свързан с роли. Според наличната реализация User има връзки със Salary, Payroll и Vacation записи, както и с бизнес представянията Employee или Manager.

5.2 Role

Role наследява Identity role и участва в ролевата сигурност на системата. Seed-натите роли са HRAdmin, Manager и Employee. Те определят кой достъпва административната зона, кой управлява отдел и кой използва само лични функционалности.

5.3 Employee

Employee съдържа организационни данни за служителя, включително позиция, връзка към User и опционална връзка към Department. DepartmentId е важен за филтрирането на служители при manager операции.

5.4 Manager

Manager е отделна entity с връзка към User. Мениджърът е свързан с Department и чрез тази връзка управлява служители в конкретен отдел. Тази структура позволява реално ограничаване на управленските права.

5.5 Department

Department съдържа име, ManagerId и колекция от Employees. Той е централна таблица за организационната логика, защото свързва мениджър, служители и операции като My Department, payroll за подчинени и одобряване на отпуск.

5.6 Salary

Salary е отделна таблица за текуща базова заплата на потребител. Отделянето ѝ от Payroll позволява историческите payroll записи да останат коректни дори при бъдеща промяна на заплата.

5.7 Payroll

Payroll съдържа Period, IssuedOn, Bonus, TaxAmount, NetAmount, GrossAmount и UserId. В текущата версия GrossAmount се базира на Salary.Amount към момента на създаване. Записът представлява финансов snapshot за конкретен период.

5.8 Vacation

Vacation съдържа StartDate, EndDate, Reason, IsApproved, IsReviewed и UserId. Двете булеви полета позволяват ясно разграничаване между чакаща, одобрена и отказана заявка.

Статус	IsReviewed	IsApproved
Pending	false	false
Approved	true	true
Rejected	true	false

6. Основни функционалности

6.1 Регистрация и вход

Потребителят може да се регистрира чрез Register форма и да влезе чрез Login. Използват се UserManager и SignInManager. След регистрация потребителят не трябва автоматично да получава пълен достъп, защото проектът моделира контролирано включване в HR система.

6.2 Одобрение на нови потребители

HRAdmin достъпва списък с чакащи потребители, преглежда данните и задава роля. Според ролята могат да се попълнят допълнителни данни като отдел и заплата. Това реализира onboarding процес, който е по-близък до реална организация.

6.3 Управление на служители

Мениджърът може да работи със служители от своя отдел. HRAdmin има по-широк административен обхват. В системата има функционалности за преглед, редакция и при определени условия изтриване, като се проверява принадлежност към отдел.

6.4 Управление на мениджъри

В административната зона HRAdmin може да преглежда и редактира мениджъри. Налична е и логика, свързана със създаване на мениджър и отдел, защото тези две понятия са тясно свързани в модела.

6.5 Управление на отдели

Отделите структурират системата. Чрез тях се свързват manager и employees. Екраните за All Departments, details и My Department дават основа за организационно управление и за правилно филтриране на данни.

6.6 Payroll функционалност

При добавяне на payroll системата извлича базовата заплата от Salary, изчислява данък 10% върху база плюс бонус и нетна сума 90%. Записът се пази като Payroll и може да се преглежда от съответния потребител.

6.7 Vacation функционалност

Employee и Manager могат да подават заявки за отпуск. Заявката първоначално е Pending. Employee заявките се разглеждат от manager на отдела, а manager заявките се разглеждат от HRAdmin. Одобрение и отказ променят IsReviewed и IsApproved.

6.8 Ролева логика

Роля	Основни действия
Employee	Личен профил, лични payroll-и, заявки за отпуск
Manager	Управление на служители в отдел, payroll операции, одобрение на employee отпуски
HRAdmin	Одобрение на потребители, управление на мениджъри и отдели, одобрение на manager отпуски

Тази таблица показва основното разделение на отговорностите. Важното е, че ролите не служат само за скриване на бутони в UI, а влияят на реалната service логика и на видимите данни.

7. Работни процеси

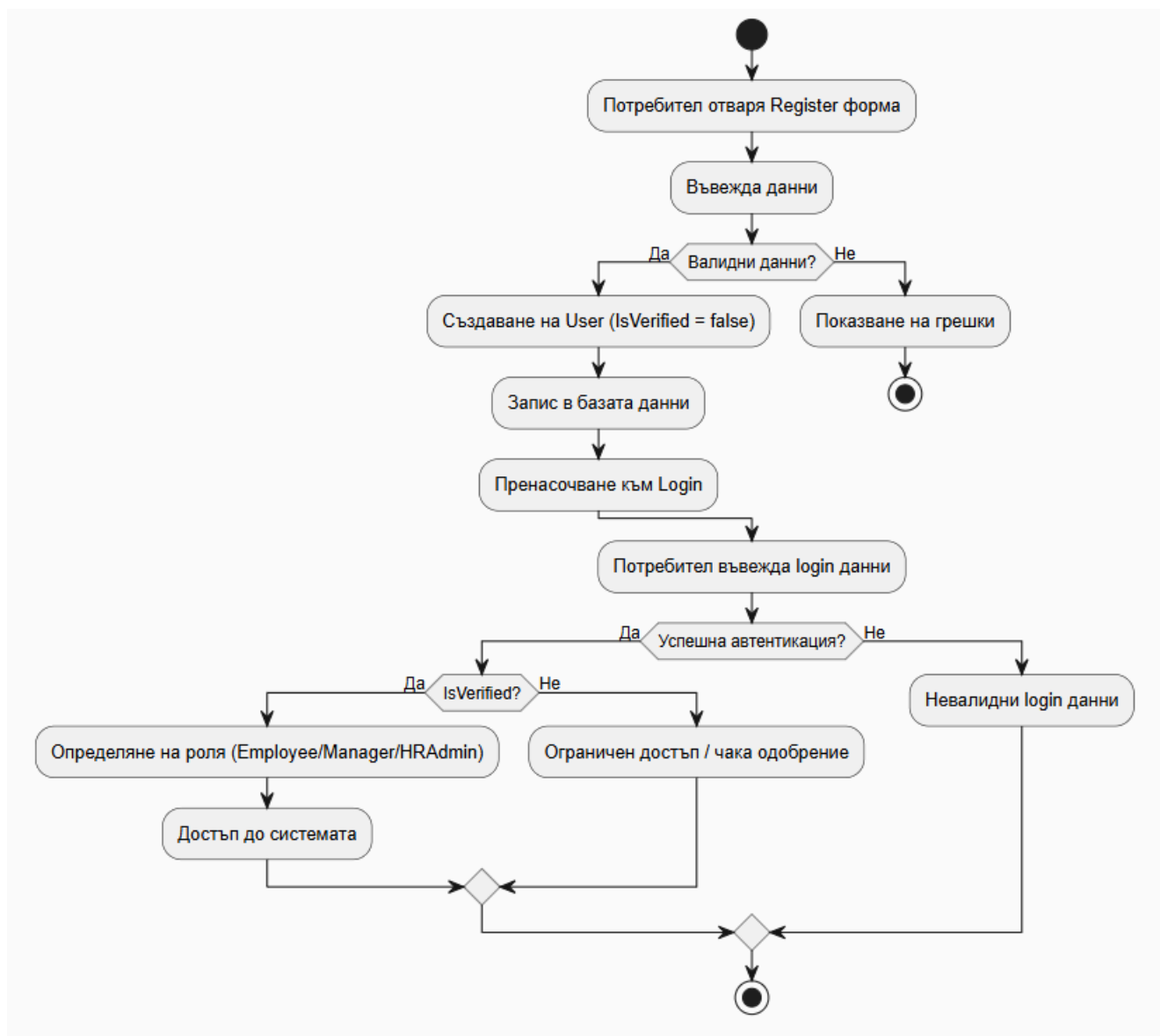
В тази глава са обобщени основните работни процеси в системата. Целта не е да се даде предимство на една функционалност, а да се покаже, че всяка ключова част от проекта следва последователна

логика: потребителско действие, controller обработка, service правила, repository достъп и запис или визуализация на резултат.

7.1 Регистрация и вход

1. Потребителят отваря Register формата и въвежда лични данни.
2. Системата създава User чрез Identity механизмите.
3. Потребителят влиза през Login формата.
4. След успешна автентикация достъпът се определя според роля и статус на профила.

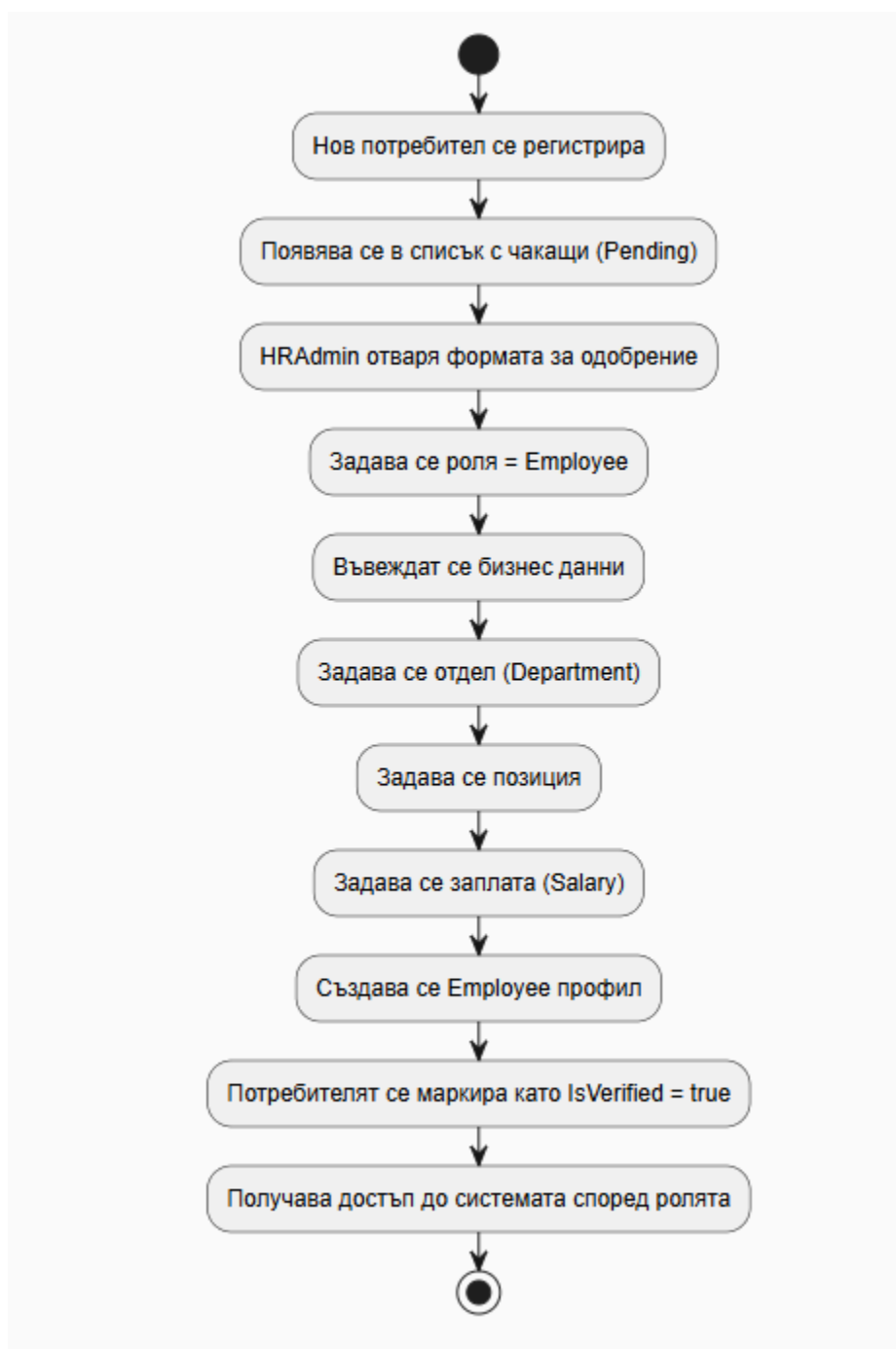
Фигура 7.1: Работен процес при регистрация и вход



7.2 Одобрение на нов потребител

1. Новият потребител се появява в списък с чакащи потребители.
2. HRAdmin отваря формата за одобрение.
3. Администраторът задава роля и нужните бизнес данни.
4. Създава се Employee или Manager профил според избора.
5. Потребителят получава реален достъп до функционалностите за ролята си.

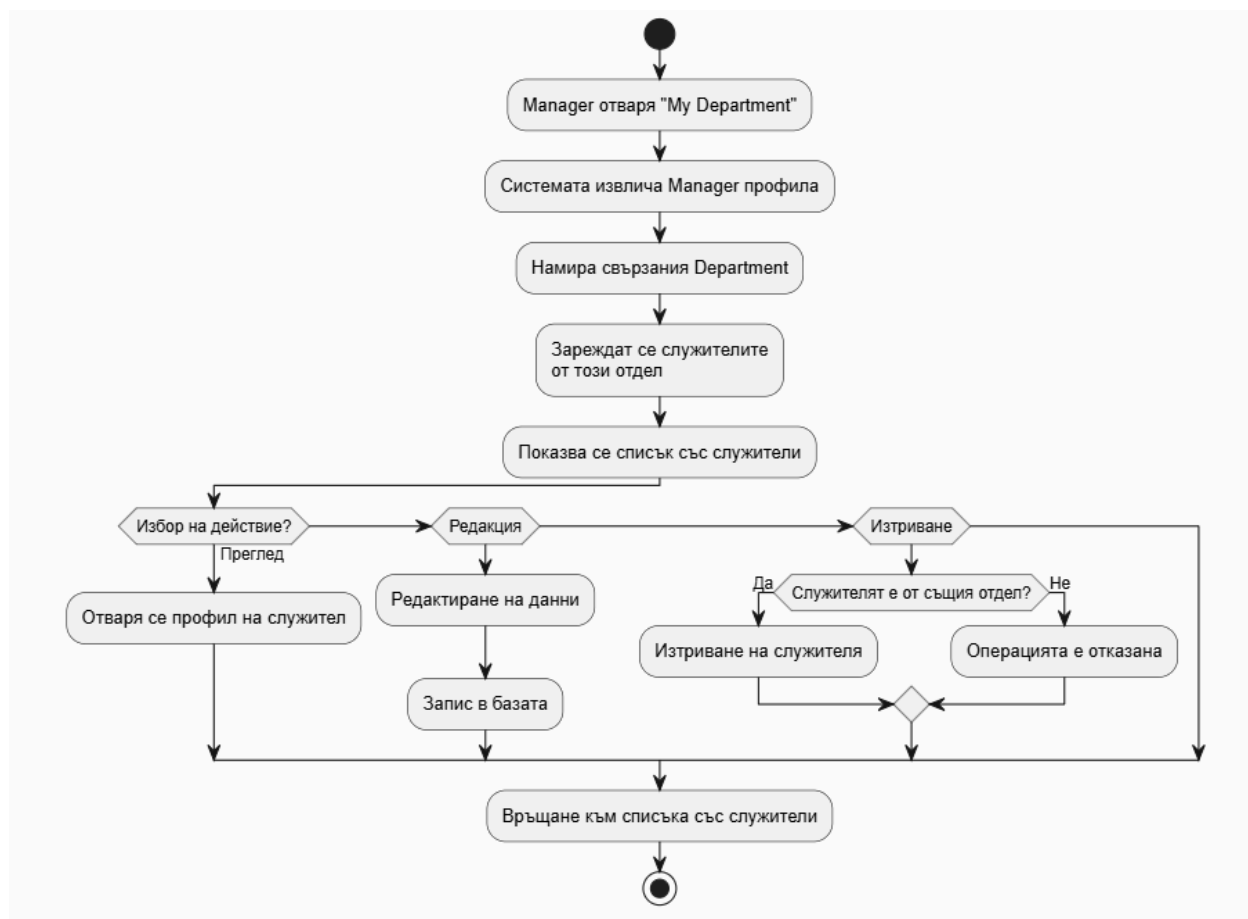
Фигура 7.2: Работен процес при одобрение на нов потребител



7.3 Управление на служители и отдели

1. Manager отваря My Department.
2. Системата намира отдела, свързан с manager профила.
3. Показват се служителите от този отдел.
4. Manager може да отвори профил, да редактира данни или да изпълни разрешени действия.
5. Service логиката предпазва от работа със служители от чужд отдел.

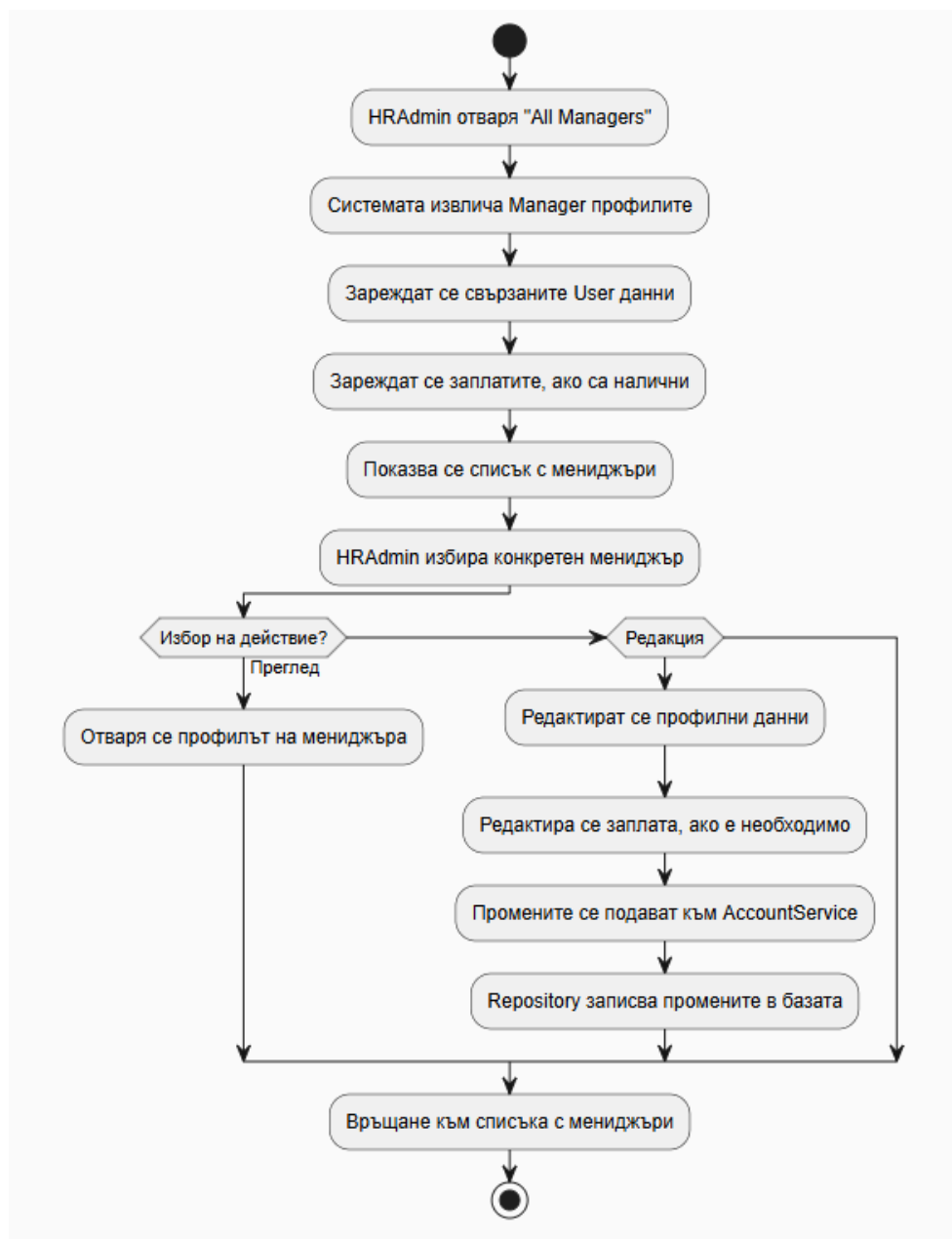
Фигура 7.3: Работен процес при управление на служители



7.4 Управление на мениджъри

1. HRAdmin отваря All Managers.
2. Системата извлича manager профилите и свързаните User данни.
3. Администраторът избира конкретен мениджър.
4. Редактират се профилни данни и заплата, ако е необходимо.
5. Промените се записват чрез AccountService и repository слоя.

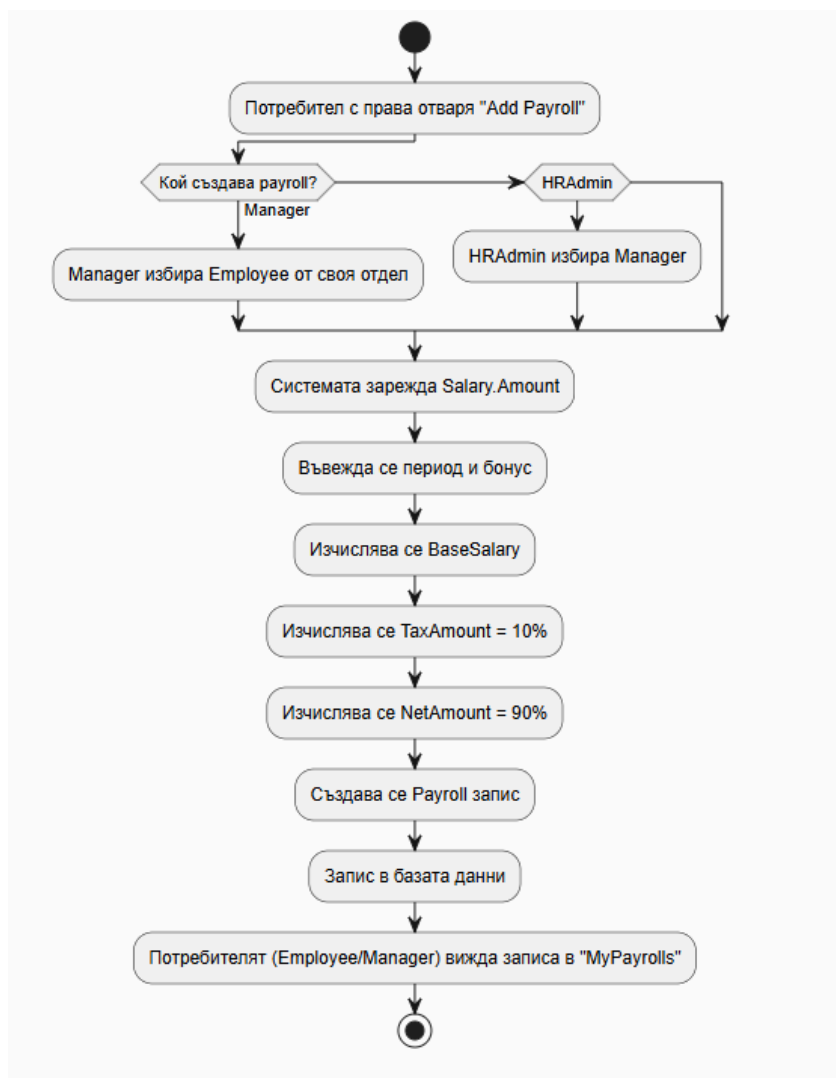
Фигура 7.4: Работен процес при управление на мениджъри



7.5 Създаване и преглед на payroll

1. Потребител с подходящи права избира служител или мениджър.
2. Системата зарежда базовата заплата от Salary.
3. Въвеждат се период и бонус.
4. PayrollService изчислява TaxAmount и NetAmount.
5. Създава се Payroll запис за потребителя.
6. Employee може да види записа в MyPayrolls.

Фигура 7.5: Работен процес при създаване на payroll

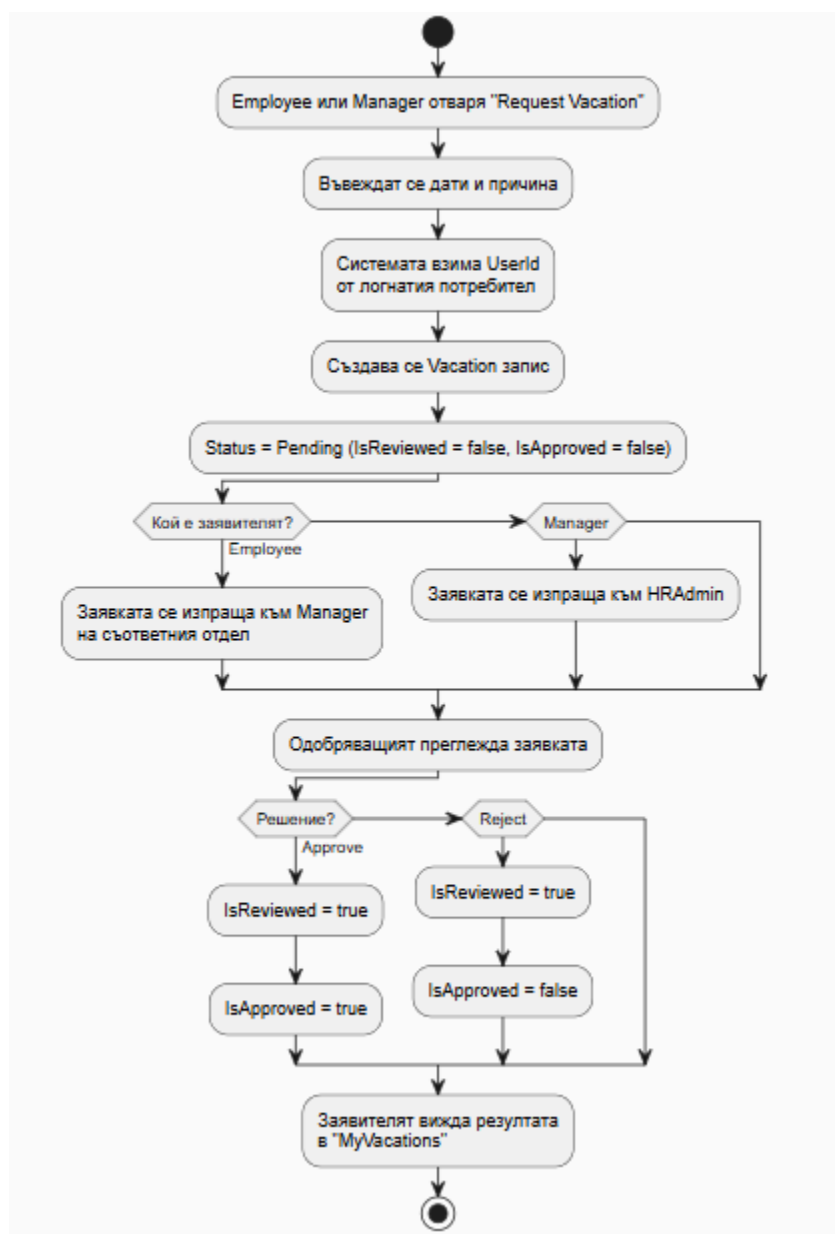


Етап	Данни	Резултат
Избор на потребител	UserId	Определя се собственикът на payroll
Зареждане на заплата	Salary.Amount	Използва се като база
Изчисления	Bonus + BaseSalary	Получават се TaxAmount и NetAmount
Запис	Payroll entity	Създава се исторически финансов запис

7.6 Заявяване и одобряване на отпуск

1. Employee или Manager отваря Request Vacation.
2. Системата записва UserId от текущия логнат потребител, а не от формата.
3. Създава се Vacation със статус Pending.
4. Ако заявителят е Employee, заявката се вижда от manager на отдела.
5. Ако заявителят е Manager, заявката се вижда от HRAdmin.
6. Approve или Reject задава IsReviewed=true и съответна стойност на IsApproved.
7. Заявителят вижда резултата в MyVacations.

Фигура 7.6: Работен процес при заявяване и одобряване на отпуск



7.7 Обобщение на процесния модел

Всички основни процеси следват сходна техническа последователност, но се различават по бизнес правилата. Регистрацията и одобрението решават достъпа до системата, управлението на отдели определя организационния обхват, payroll логиката обработва финансови записи, а отпуските въвеждат статусен approval процес. Това прави проекта достатъчно последователен, без отделна функционалност да изглежда изолирана от останалите.

8. Описание на реализацията

8.1 Program.cs

Program.cs е композиционната точка на приложението. В него се регистрират DbContext, Identity, repository и business services. Така ASP.NET Core контейнерът може да подава нужните зависимости към контролерите.

Код 8.1: Регистрация на ключови зависимости

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddScoped<IRepository, Repository>();
builder.Services.AddScoped<IAccountService, AccountService>();
builder.Services.AddScoped<IRoleService, RoleService>();
builder.Services.AddScoped<IVacationService, VacationService>();
builder.Services.AddScoped<IPayrollService, PayrollService>();
builder.Services.AddScoped<IDepartmentService, DepartmentService>();
```

8.2 DbContext

ApplicationDbContext обединява Identity таблиците и бизнес таблиците. В OnModelCreating се прилагат configuration класове за роли, потребители, отдели, служители, мениджъри, заплати, payroll и отпуски.

Код 8.2: Прилагане на конфигурации и seed

```
builder.ApplyConfiguration(new RoleConfiguration());
builder.ApplyConfiguration(new UserConfiguration());
builder.ApplyConfiguration(new DepartmentConfiguration());
builder.ApplyConfiguration(new ManagerConfiguration());
builder.ApplyConfiguration(new EmployeeConfiguration());
builder.ApplyConfiguration(new SalaryConfiguration());
builder.ApplyConfiguration(new PayrollConfiguration());
builder.ApplyConfiguration(new VacationConfiguration());
```

8.3 Repository

Repository предоставя общ интерфейс за достъп до данните. Чрез него services могат да четат и променят записи, без да създават директна зависимост към DbContext във всеки метод.

Код 8.3: Основни операции за достъп до данни

```
public IQueryable<T> All<T>() where T : class => DbSet<T>().AsQueryable();

public IQueryable<T> AllReadonly<T>() where T : class
```

```

=> this.DbSet<T>().AsNoTracking();

public async Task AddAsync<T>(T entity) where T : class
=> await DbSet<T>().AddAsync(entity);

public async Task<T> GetByIdAsync<T>(object id) where T : class
=> await DbSet<T>().FindAsync(id);

public async Task<int> SaveChangesAsync()
=> await this.Context.SaveChangesAsync();

```

8.4 AccountService

AccountService подготвя данни за потребители, профили, служители и административни операции. В него се среща ролево филтриране, например при извличане на потребители без HRAdmin.

Код 8.4: Филтриране на потребители без HRAdmin

```

var users = await (
    from user in repo.All<User>()
    join ur in repo.All<UserRole>() on user.Id equals ur.UserId
    join role in repo.All<Role>() on ur.RoleId equals role.Id
    where role.Name != "HRAdmin"
    select new DisplayedUserModel
    {
        UserId = user.Id,
        FirstName = user.FirstName,
        LastName = user.LastName,
        Email = user.Email,
        IsVerified = user.IsVerified
    }).ToListAsync();

```

8.5 PayrollService

PayrollService съдържа финансовата логика. При създаване на payroll се взима базовата заплата, добавя се бонус и се изчисляват данък и нетна сума. Така в записа се пази резултат за конкретен период.

Код 8.5: Изчисления при създаване на payroll

```

model.BaseSalary = salary.Amount;
model.TaxAmount = (model.Bonus + model.BaseSalary) * 0.1m;
model.NetAmount = (model.Bonus + model.BaseSalary) * 0.9m;

```

8.6 VacationService

VacationService обработва заявяване, преглед и одобряване на отпуски. При pending заявки service-ът различава Manager и HRAdmin гледна точка. Manager вижда заявки от служители в своя отдел, а HRAdmin вижда заявки от manager потребители.

Код 8.6: Role-based одобрение на pending отпуски

```

if (!isHrAdmin)
{
    var employeeUserIds = await repository
        .AllReadOnly<Employee>()
        .Where(e => e.DepartmentId == departmentId)

```

```

        .Select(e => e.UserId)
        .ToListAsync();

    query = query.Where(v => employeeUserIds.Contains(v.UserId));
}
else
{
    var managerUserIds = await repository
        .AllReadonly<Manager>()
        .Select(m => m.UserId)
        .ToListAsync();

    query = query.Where(v => managerUserIds.Contains(v.UserId));
}

```

8.7 Controllers

Контролерите реализират входните точки към функционалностите. AccountController обработва вход, регистрация и профили. PayrollController управлява преглед и добавяне на payroll. VacationController обработва заявяване, преглед и approve/reject на отпуски. В Areas/Admin са отделени административните операции.

8.8 ViewModels

ViewModel-ите са отделени от entity моделите, за да не се излага директно структурата на базата към UI. Примери са PayrollViewModel, VacationViewModel, VacationFormModel, DisplayedEmployeeModel и ApproveUserInputModel. Това улеснява валидацията и прави изгледите по-ясни.

8.9 Razor Views

Razor изгледите реализират формите, таблиците и визуализацията на статуси. При payroll и vacation модулите се използват форми за въвеждане и отделни изгледи за списъци. Status badges и бутоните за approve/reject дават ясна обратна връзка на потребителя.

9. Примери за използване

9.1 Нов потребител се регистрира

Потребителят попълва регистрационната форма. Системата създава Identity потребител и го насочва към login. До одобрение от HRAdmin достъпът остава ограничен според текущата реализация.

9.2 HR Admin одобрява потребител

HRAdmin отваря списък с чакащи потребители, избира конкретен запис и задава роля, отдел и начална заплата според нужния бизнес профил. След това потребителят може да работи с функционалности според ролята си.

9.3 Manager управлява служители от своя отдел

Manager достъпва My Department и вижда служителите в своя отдел. При редакция или изтриване системата проверява принадлежността към отдела, което предпазва от работа с чужди служители.

9.4 Manager създава payroll

Мениджърът избира служител, попълва период и бонус, а системата извлича базовата заплата от Salary и изчислява TaxAmount и NetAmount. След запис служителят вижда новия payroll в личния си списък.

9.5 Employee преглежда payroll

Employee отваря MyPayrolls и вижда само собствените си payroll записи, филтрирани по текущ UserId. Това показва личния финансов изглед без достъп до чужди данни.

9.6 Employee заявява отпуск

Employee отваря RequestVacation, попълва начална дата, крайна дата и причина. След submit се създава Vacation запис със статус Pending и заявката се появява в MyVacations.

9.7 Manager одобрява или отказва отпуск

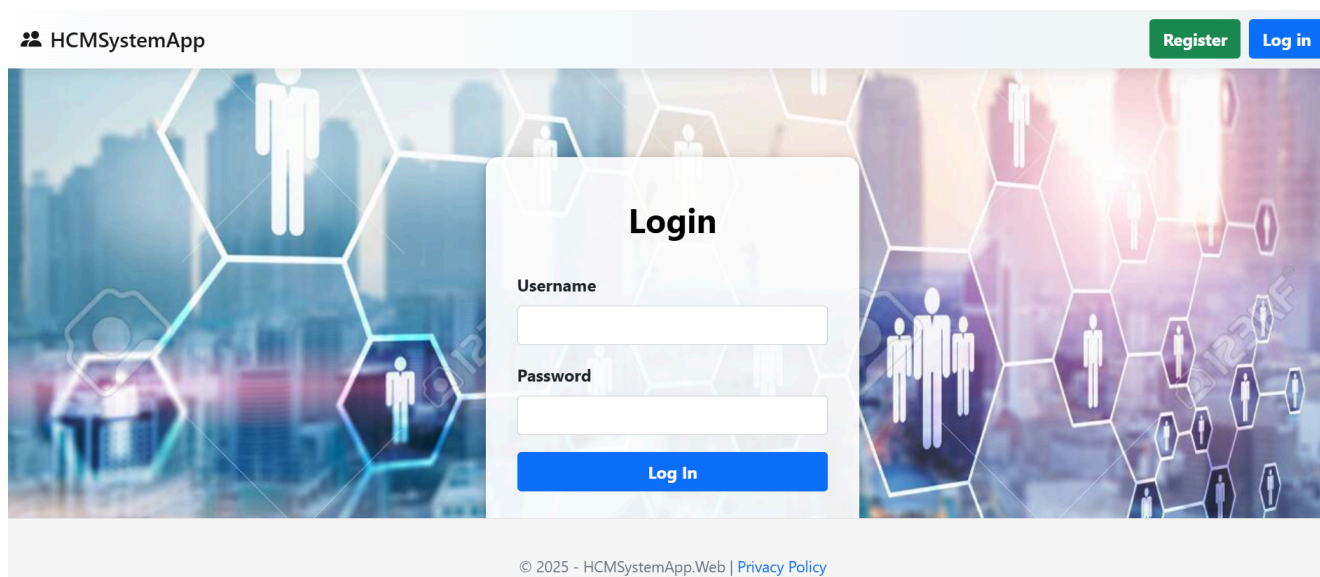
Manager отваря PendingVacations и вижда заявки от служители в своя отдел. При Approve заявката става Approved, а при Reject става Rejected. И в двата случая тя вече не е pending.

9.8 Manager заявява отпуск и HR Admin го одобрява

Manager може също да подаде заявка за отпуск. Тази заявка не се разглежда от друг manager, а от HRAdmin. Така се избягва самоодобрение и се поддържа логична административна отговорност.

10. Екрани за демонстрация

Фигура 10.1: Login страница



Екранът показва входа в системата. Чрез него се демонстрира Identity authentication и началната точка за достъп до ролевите функционалности.

Фигура 10.2: Register страница

HCMSystemApp

Register Log In

Register

UserName

Email

FirstName

LastName

PhoneNumber

Age

Password

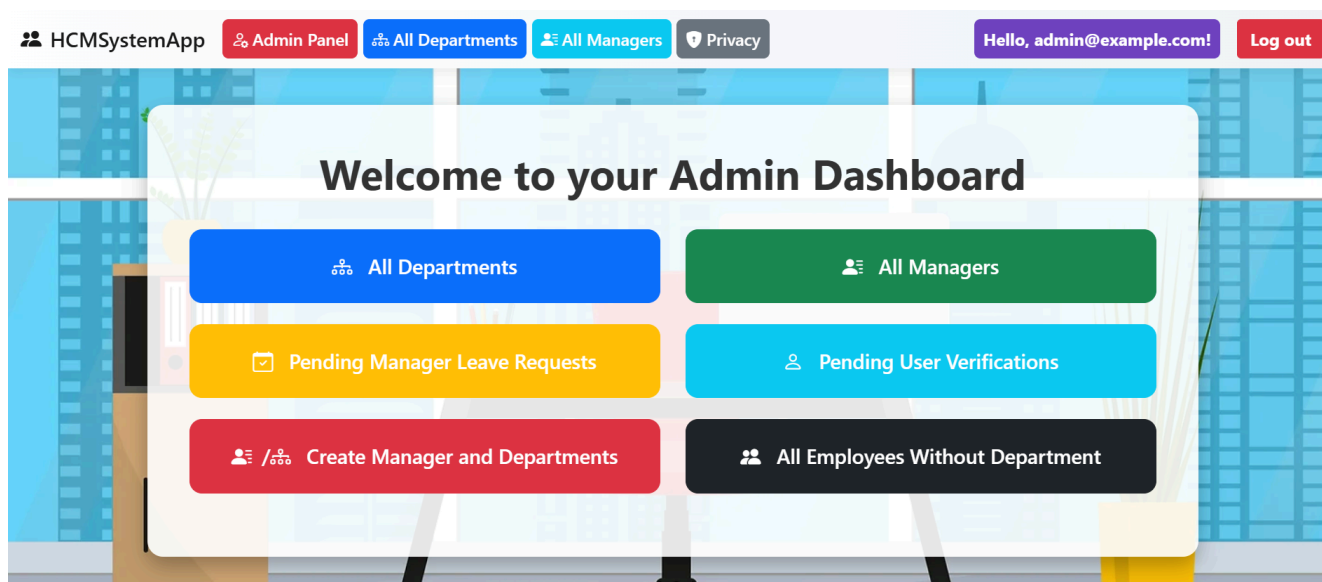
ConfirmPassword

Register

© 2023 - HCMSystemApp Web | Privacy Policy

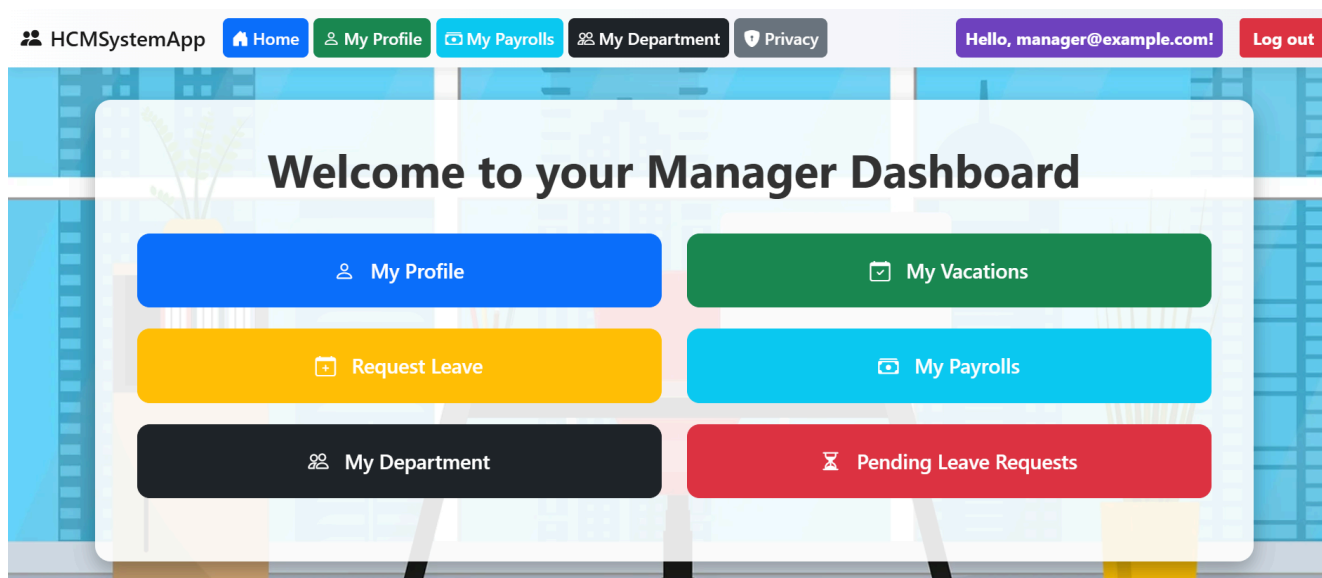
Тук се показва създаването на нов потребител. Екранът е важен за демонстрация на onboarding процеса преди HR одобрение.

Фигура 10.3: Admin Dashboard



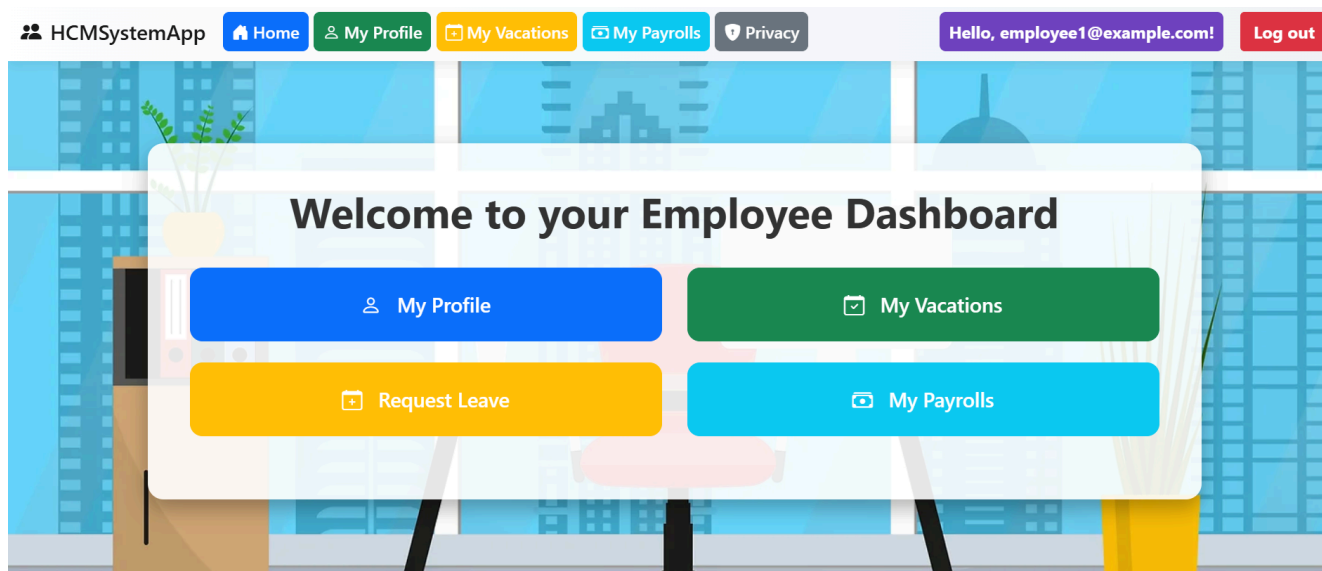
Административният dashboard обединява действията за HRAdmin. Чрез него се вижда разделението между обикновен потребител и администратор.

Фигура 10.4: Manager Dashboard



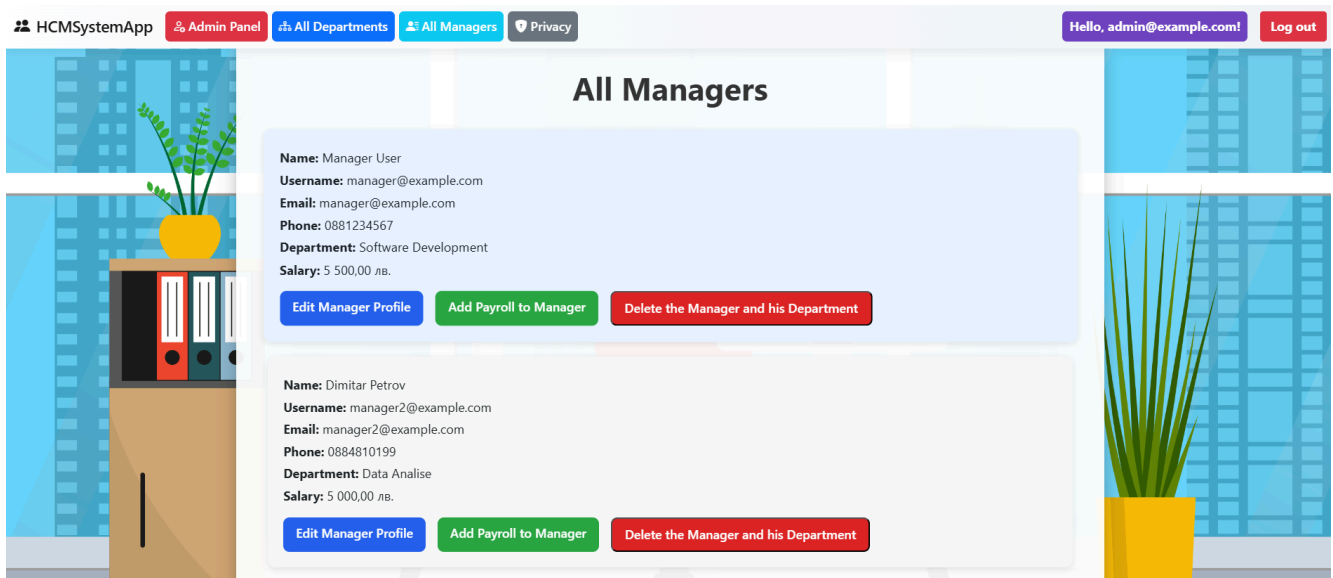
Manager dashboard показва действията, свързани с отдел, служители и approval процеси. Той демонстрира ролята на department-based достъп.

Фигура 10.5: Employee Dashboard



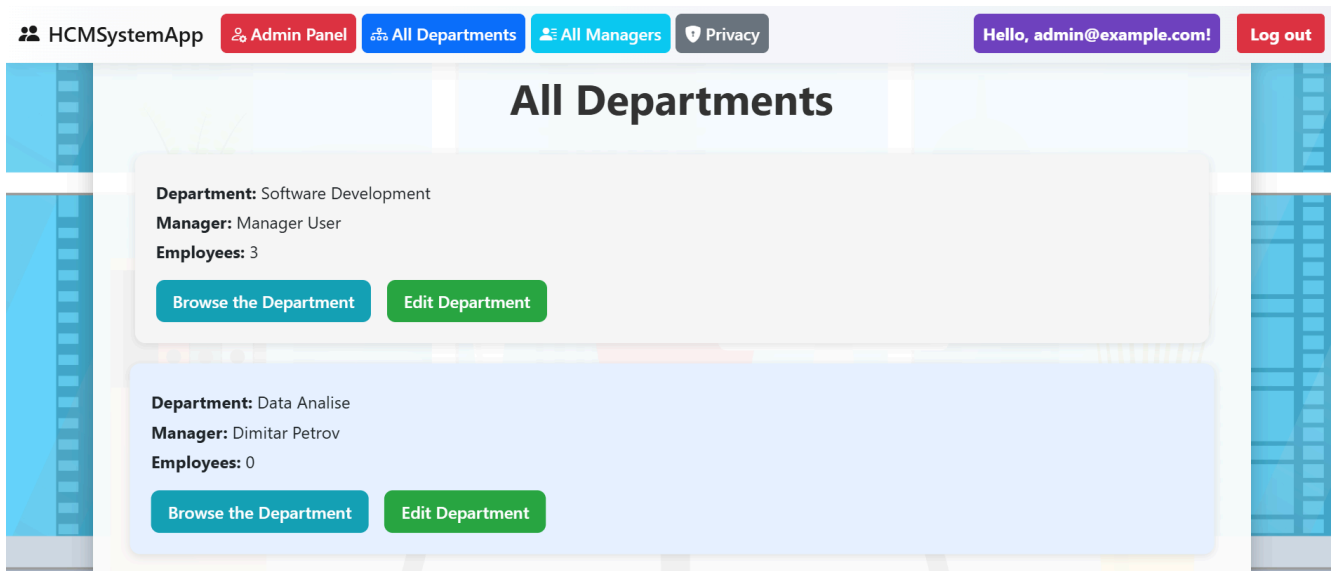
Employee dashboard представя личния потребителски контекст: профил, payroll и отпуски.

Фигура 10.6: All Managers



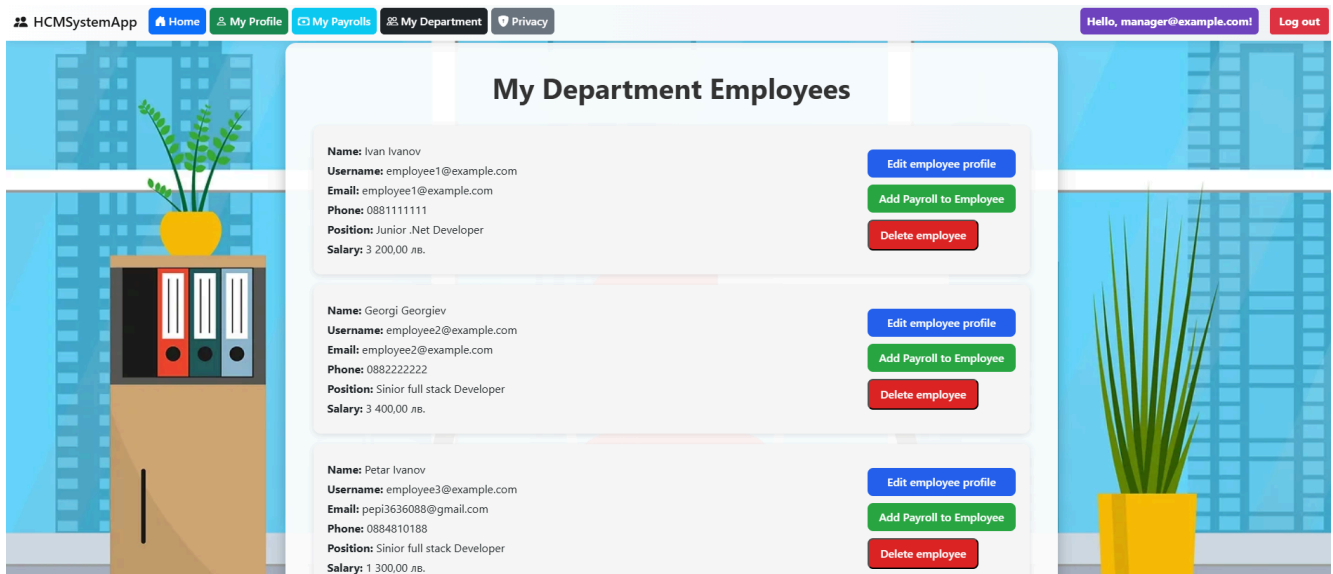
Екранът показва списък с мениджъри и е част от административната поддръжка на организацията.

Фигура 10.7: All Departments



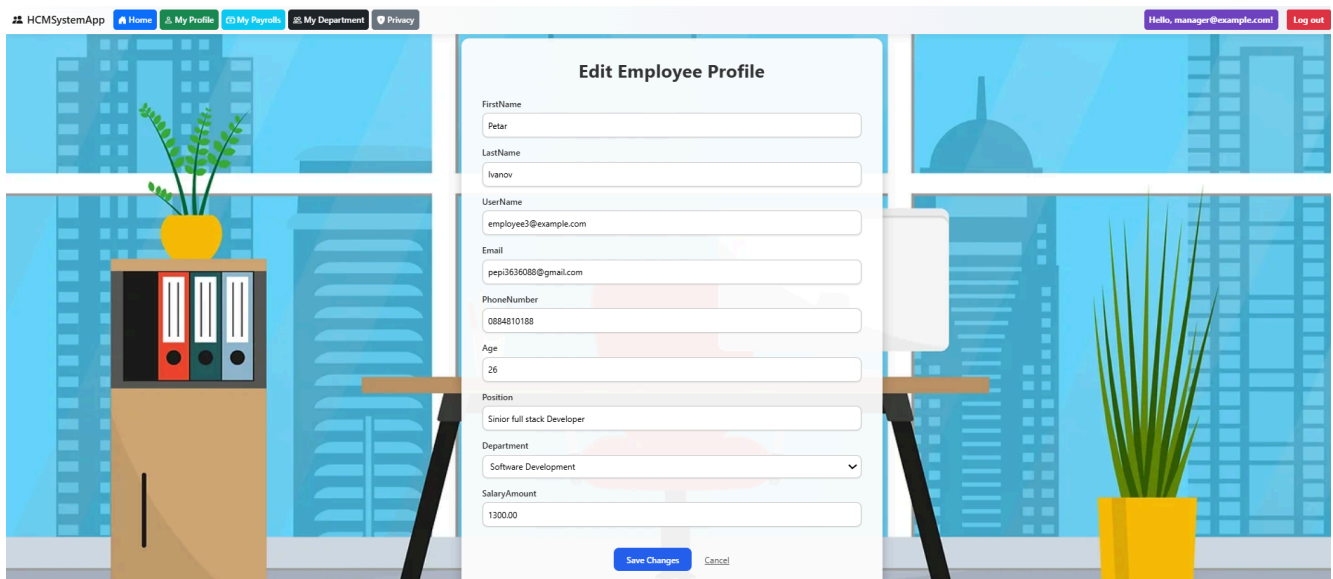
Този екран показва организационната структура чрез отдели, свързани мениджъри и служители.

Фигура 10.8: My Department



My Department показва служителите, които конкретният manager може да управлява.

Фигура 10.9: Edit Employee Profile



Екранът показва редакция на служителски данни, включително профилна информация и заплата, според правата на текущия потребител.

Фигура 10.10: Add Payroll

HCMSystemApp Home My Profile My Payrolls My Department Privacy Hello, manager@example.com! Log out

Add Payroll

Base Salary
1 300,00 лв.

Period
e.g. July 2025

IssuedOn
05/02/2026

Bonus
0

Create Payroll

Cancel

© 2025 - HCMSystemApp.Web | Privacy Policy

Тук се демонстрира създаване на payroll запис с период, бонус и автоматични изчисления.

Фигура 10.11: My Payrolls

HCMSystemApp Home My Profile My Vacations My Payrolls Privacy Hello, employee1@example.com! Log out

My Payrolls

Period: 01-2025
Issued On: 31.1.2025 г.
Salary Amount: 3 200,00 лв.
Bonus: 200,00 лв.
Tax: 480,00 лв.
Net Amount: 2 920,00 лв.

Period: 02-2025
Issued On: 28.2.2025 г.
Salary Amount: 3 200,00 лв.
Bonus: 150,00 лв.
Tax: 460,00 лв.
Net Amount: 2 890,00 лв.

Личният payroll екран показва историческите записи за текущия потребител.

Фигура 10.12: Request Vacation

Request Vacation

StartDate: 05/03/2026

EndDate: 05/06/2026

Reason: Reason for vacation request

Submit Request

Cancel

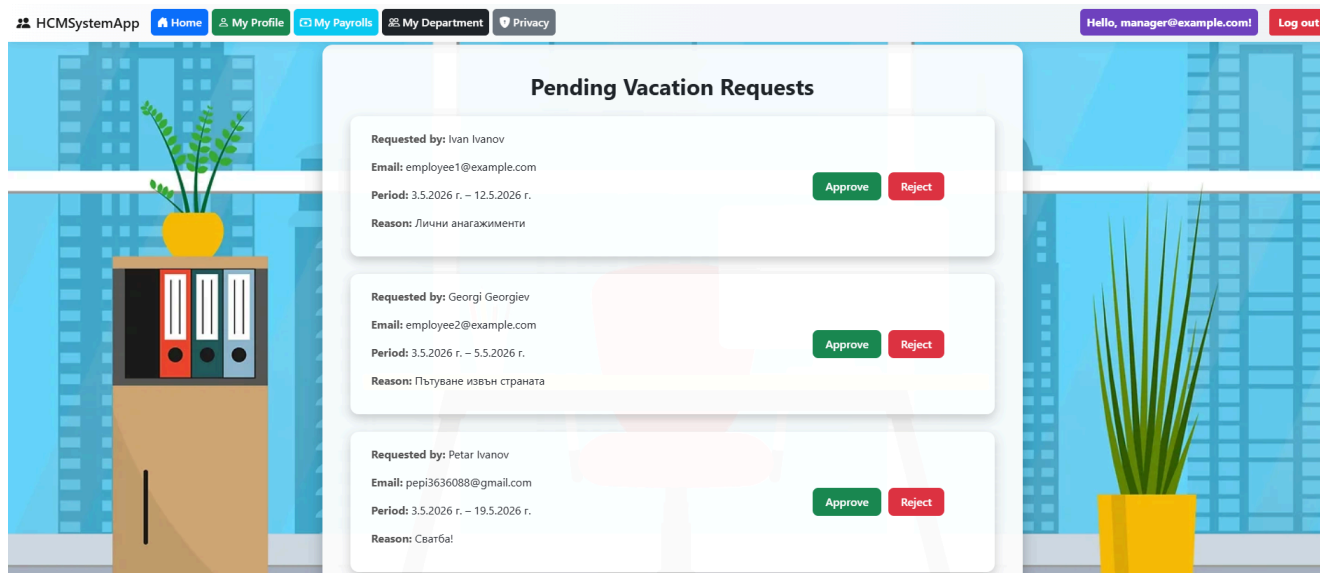
Формата за отпуск показва заявяване на нова отпуска и началото на approval процеса.

Фигура 10.13: My Vacations

My Vacation Requests	
Period: 10.5.2025 г. – 17.5.2025 г. Reason: Почивка в Гърция	Reviewed: Yes Approved: Approved
Period: 12.8.2025 г. – 19.8.2025 г. Reason: Екскурзия до Италия	Reviewed: Yes Approved: Approved
Period: 28.4.2026 г. – 1.5.2026 г. Reason: Да отида на море	Reviewed: Yes Approved: Rejected

Този екран показва личните отпуска и техните статуси Pending, Approved или Rejected.

Фигура 10.14: Pending Vacation Requests



Екранът показва чакащи заявки за одобрение според ролята: manager вижда служители от отдел, а HRAdmin вижда заявки на manager-и.

11. Трудности и преодоленни проблеми

При разработката на проекта могат да се посочат няколко технически предизвикателства. Първото е конфигурирането на Identity с custom User, Role и UserRole, така че стандартната authentication логика да работи заедно с бизнес модела.

Второто предизвикателство е поддържането на коректни seed зависимости. Ролите трябва да съществуват преди UserRole записите, потребителите трябва да съществуват преди Employee и Manager профилите, а Salary и Payroll зависят от коректни User връзки.

Трети проблем е разграничаването между User, Employee и Manager. Това решение е необходимо, защото User е техническа identity единица, докато Employee и Manager имат бизнес значение. Разделението намалява смесването на отговорности.

Ограничаването на manager само до собствения му отдел също е важна част. Това е решено чрез Department и Employee връзките и чрез service филтриране. Така role-based access не остава само на ниво UI, а влияе реално върху данните.

Отделянето на Salary от Payroll е друго важно решение. Salary представя текуща стойност, а Payroll е исторически запис за период. Това позволява финансовите записи да запазят смисъл при бъдещи промени в заплатите.

При отпуските основното предизвикателство е статусният модел. Използването на IsReviewed и IsApproved позволява ясно разграничение между Pending, Approved и Rejected. Така отказаната заявка не се бърка с такава, която още не е разгледана.

Накрая, организацията на service слоя предотвратява натрупване на бизнес логика в контролерите. Контролерите остават отговорни за HTTP потока, а service класовете съдържат реалните правила.

12. Заключение

HCMSystemApp представлява завършено уеб приложение за управление на човешки ресурси, изградено с ASP.NET Core MVC, Entity Framework Core и ASP.NET Core Identity. Проектът използва слоеста архитектура, която позволява по-добра поддръжка, разделяне на отговорности и бъдещо разширяване.

Системата покрива регистрация и одобрение на потребители, ролева организация, управление на отдели, служители и мениджъри, payroll процеси и управление на отпуски. Това я прави по-богата от стандартен CRUD проект, защото включва контекстно поведение според роля, организационни зависимости и работни процеси.

Възможни бъдещи подобрения са разширени справки, export на payroll документи, календарен изглед на отпуските, email известия, audit log за ключови действия, допълнителна responsive оптимизация и отделен API слой за интеграции. Тези посоки показват, че проектът може да бъде надграден към още по-практична HCM система.

13. Литература

1. Microsoft Learn – ASP.NET Core MVC documentation.
2. Microsoft Learn – Entity Framework Core documentation.
3. Microsoft Learn – ASP.NET Core Identity documentation.
4. Microsoft Learn – Dependency Injection in ASP.NET Core.
5. Microsoft SQL Server documentation.
6. Bootstrap documentation.
7. GitHub repository: <https://github.com/PetarIvanov33/HCMSystemApp>